

# FRED TALKS

23<sup>rd</sup> September 2026

---

## CONTENTS

Headless everything for personal AI

INTERCONNECTED · 1335 WORDS

---

Highlights from my conversation about agentic engineering on  
Lenny's Podcast

SIMON WILLISON'S WEBLOG: ENTRIES · 3349 WORDS

---

the senior engineer death spiral

SUNIL PAI · 959 WORDS

---

[Daily article] September 23: Mary Mallon

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 471 WORDS

---

# Headless everything for personal AI

INTERCONNECTED · 18 APR 2026 · [SOURCE](#)

---

It's pretty clear that apps and services are all going to have to go *headless*: that is, they will have to provide access and tools for personal AI agents without any of the visual UI that us humans use today.

By services I mean things like: getting a new passport; finding and booking a hotel or a flight; managing your bank account; shopping for t-shirts with a minimum cotton weight from brands similar to ones that you've bought from before.

Why? Because using personal AIs is a better experience for users than using services directly (honestly); and headless services are quicker and more dependable for the personal AIs than having them click round a GUI with a bot-controlled mouse.

Where this leaves design for the services... well, I have some thoughts on that.

Headless services? They're happening already.

Already there is [MCP, as previously discussed](#) (2025), a kind of web API specifically for AI. For instance, best-in-class call transcriber app Granola [recently released their MCP](#) and now you can ask your Claude to pull out the meeting actions and go trawling in your personal docs to find answers to all the question. A good integration.

But command-line tools are growing in popularity, although they used to be just for developers. So you can now create a spreadsheet by typing in your terminal:

```
gws sheets spreadsheets create --json '{"properties": {"title":  
"Q1 Budget"}}'
```

Here are some recently launched CLI tools:

- [gws](#): Google Workspace CLI, Drive, Gmail, Calendar, and every Workspace API. Zero boilerplate. Structured JSON output. 40+ agent skills included.
- [Obsidian CLI](#) to do anything you can do with the (very popular) note-taking app - like keep daily notes, track and cross off tasks, search - from the CLI

- Salesforce CLI and, look, I don't really understand what the Salesforce business operating system does, but the fact it has a CLI too is significant.

And here is CLI-Anything (31k stars on GitHub) which auto-generates CLIs for ANY codebase.

Why CLIs?

It turns out that the best place for personal AIs to run is on a computer. Maybe a virtual computer in the cloud, but ideally your computer. That way they can see the docs that you can see, and use the tools that you can use, and so what they want is not APIs (which connect webserver) but little apps they can use directly. CLI tools are the perfect little apps.

CLIs are composable so they are a better fit for what users actually do.

By composable I mean you can: query your notes then jump to a spreadsheet then research the web then jump back to the spreadsheet then text the user a clarifying question then double-check your notes, all by bouncing between CLIs in the same session.

A while back app design got obsessed with “user journeys.” Like the journey of a user finding a hotel then booking a hotel then staying in it and leaving a review.

But users don't live in “journeys.” They multitask; they talk to people and come back to things; they're idiosyncratic Try grabbing the search results from the Airbnb app and popping them in a message to chat on the family WhatsApp, then coming back to it two days later. It's a pain and you have to use screenshots because apps and their user journeys are not composable.

CLIs are composable because they came originally from Unix and that is the Unix tools philosophy: tools were designed so that they could operate together.

Personal AIs like Openclaw or [Poke](<https://poke.com>) do what users want and don't follow “designed” user journeys, and as a result the composed experience is more personal and way better. CLIs are a great underlying technology for that.

CLIs are smaller than regular apps and so they are easier to secure.

The alternative to providing special tools for AIs is that the AIs use the same browser-based apps that we do, and that's a terrifying prospect.

For one, AIs are really good at finding security holes. The new Mythos model from Anthropic is so good at discovering security flaws that it has been held back from the public and governments are convening emergency meetings of the biggest banks.

And including a UI increasing complexity and makes security holes more likely.

Here's a recent shocking example. Companies House is the national register of all companies, directors, and accounts for England and Wales. Users could view *and edit* any other user's account:

a logged-in company director could exploit the flaw by starting from their own dashboard and then trying to log into another company's account.

Once they reach the 2FA block, which they would not be able to pass, all that was required was to click the browser's back button a few times. Typically, the user would be taken back to their own dashboard, but the bug instead returned them to the company they had tried to log into but couldn't.

This bug had been present since October 2025.

Imagine a future where personal AIs are filing company records, and one of them notices this security hole overnight and posts about it on moltbook or whatever agent-only social network is most popular. The other agents would exploit the system up, down and sideways before the engineering team woke up.

The only viable solution is that services need to security hardened, and to do that they need to be simplifying and minimised. Again, CLI tools are a great fit.

What does this mean for front-end design?

Design won't go anywhere.

Sure, the front-end should be driving the same CLI tools that agents use.

Arguably it's more important than ever: human users will encounter services, figure out what they can do, and pick up their vibe from using the app, just as now.

Then they'll tell their personal AI about the service and never see the front-end again, or re-mix it into bespoke personal software.

So from a usability perspective I see front-end as somewhat sacrificial. AI agents will drive straight through it; users will encounter it only once or twice; it will be customised or personalised; all that work on optimising user journeys doesn't matter any more.

But from a vibe perspective, services are not fungible. e.g. if you're finding a restaurant then Yelp, Google Maps, Resy, and The Infatuation are all more-or-less equivalent for answering that question but clearly they are completely different and you'll use different services at different times.

Understanding that a service is *for you* is 50% an unconscious process - we call it brand - and I look forward to front-end design for apps and services optimising for brand rather than ease of use.

If I were a bank, I would be releasing a hardened CLI tool like *yesterday*.

There is so much to figure out:

- How do permissions work? Should the user get a notification from their phone app when the agent strays outside normal behaviour? How do I give it credentials to act on my behalf, and how long do they last?
- How does adjacency work? My bank gives me a current account in exchange for putting a "hey, get a loan!" button on the app home screen. How do you make offers to an agent?

Headless banks.

Headless government?

I'd love to show you a worked example here. I vibed up a suite of four CLI tools that wrap four different services from UK government departments.

If I were renting a house, I would set my agent to learning about the neighbourhoods using one of these tools. Another tool will be helpful next time I'm buying a used car. (There's a Companies House command-line tool too.)

But I won't show you the tools because I don't want to be responsible for maintaining and supporting them.

I wish Monzo came with an official CLI. I wish Booking.com came with a CLI. I reckon, give it a year, they will.

**Auto-calculated kinda related posts:**

If you enjoyed this post, please consider sharing it by email or on social media. [Here's the link.](#)  
Thanks, —Matt.

---

# Highlights from my conversation about agentic engineering on Lenny's Podcast

SIMON WILLISON'S WEBLOG: ENTRIES · 02 APR 2026 · [SOURCE](#)

---

## Highlights from my conversation about agentic engineering on Lenny's Podcast

I was a guest on Lenny Rachitsky's podcast, in a new episode titled [An AI state of the union: We've passed the inflection point, dark factories are coming, and automation timelines](#). It's available on [YouTube](#), [Spotify](#), and [Apple Podcasts](#). Here are my highlights from our conversation, with relevant links.

[An AI state of the union: We've passed the inflection point & dark factories are coming](#)[Lenny's Podcast](#)



Lenny's Podcast 571K subscribers

[Watch on](#)

- [The November inflection point](#)
- [Software engineers as bellwethers for other information workers](#)
- [Writing code on my phone](#)
- [Responsible vibe coding](#)
- [Dark Factories and StrongDM](#)
- [The bottleneck has moved to testing](#)
- [This stuff is exhausting](#)

- Interruptions cost a lot less now
- My ability to estimate software is broken
- It's tough for people in the middle
- It's harder to evaluate software
- The misconception that AI tools are easy
- Coding agents are useful for security research now
- OpenClaw
- Journalists are good at dealing with unreliable sources
- The pelican benchmark
- And finally, some good news about parrots
- YouTube chapters

## The November inflection point #

4:19—The end result of these two labs throwing everything they had at making their models better at code is that in November we had what I call the inflection point where GPT 5.1 and Claude Opus 4.5 came along.

They were both incrementally better than the previous models, but in a way that crossed a threshold where previously the code would mostly work, but you had to pay very close attention to it. And suddenly we went from that to... almost all of the time it does what you told it to do, which makes all of the difference in the world.

Now you can spin up a coding agent and say, build me a Mac application that does this thing, and you'll get something back which won't just be a buggy pile of rubbish that doesn't do anything.

# Software engineers as bellwethers for other information workers #

5:49—I can churn out 10,000 lines of code in a day. And most of it works. Is that good? Like, how do we get from most of it works to all of it works? There are so many new questions that we’re facing, which I think makes us a bellwether for other information workers.

Code is easier than almost every other problem that you pose these agents because code is obviously right or wrong—either it works or it doesn’t work. There might be a few subtle hidden bugs, but generally you can tell if the thing actually works.

If it writes you an essay, if it prepares a lawsuit for you, it’s so much harder to derive if it’s actually done a good job, and to figure out if it got things right or wrong. But it’s happening to us as software engineers. It came for us first.

And we’re figuring out, OK, what do our careers look like? How do we work as teams when part of what we did that used to take most of the time doesn’t take most of the time anymore? What does that look like? And it’s going to be very interesting seeing how this rolls out to other information work in the future.

Lawyers are falling for this really badly. The [AI hallucination cases database](#) is up to 1,228 cases now!

Plus this bit from the cold open at [the start](#):

It used to be you’d ask ChatGPT for some code, and it would spit out some code, and you’d have to run it and test it. The coding agents take that step for you now. And an open question for me is how many other knowledge work fields are actually prone to these agent loops?

## Writing code on my phone #

8:19—I write so much of my code on my phone. It’s wild. I can get good work done walking the dog along the beach, which is delightful.

I mainly use the Claude iPhone app for this, both with a regular Claude chat session (which [can execute code now](#)) or using it to control [Claude Code for web](#).

## Responsible vibe coding #

9:55 If you're vibe coding something for yourself, where the only person who gets hurt if it has bugs is you, go wild. That's completely fine. The moment you ship your vibe coding code for other people to use, where your bugs might actually harm somebody else, that's when you need to take a step back.

See also [When is it OK to vibe code?](#)

## Dark Factories and StrongDM #

12:49 The reason it's called the dark factory is there's this idea in factory automation that if your factory is so automated that you don't need any people there, you can turn the lights off. Like the machines can operate in complete darkness if you don't need people on the factory floor. What does that look like for software? [...]

So there's this policy that nobody writes any code: you cannot type code into a computer. And honestly, six months ago, I thought that was crazy. And today, probably 95% of the code that I produce, I didn't type myself. That world is practical already because the latest models are good enough that you can tell them to rename that variable and refactor and add this line there... and they'll just do it—it's faster than you typing on the keyboard yourself.

The next rule though, is nobody *reads* the code. And this is the thing which StrongDM started doing last year.

I wrote a lot more about [StrongDM's dark factory explorations](#) back in February.

## The bottleneck has moved to testing #

21:27—It used to be, you'd come up with a spec and you hand it to your engineering team. And three weeks later, if you're lucky, they'd come back with an implementation. And now that maybe takes three hours, depending on how well the coding agents are established for that kind of thing. So now what, right? Now, where else are the bottlenecks?

Anyone who's done any product work knows that your initial ideas are always wrong. What matters is proving them, and testing them.

We can test things so much faster now because we can build workable prototypes so much quicker. So there's an interesting thing I've been doing in my own work where any feature that I want to design, I'll often prototype three different ways it could work because that takes very little time.

I've always loved prototyping things, and prototyping is even more valuable now.

22:40—A UI prototype is free now. ChatGPT and Claude will just build you a very convincing UI for anything that you describe. And that's how you should be working. I think anyone who's doing product design and isn't vibe coding little prototypes is missing out on the most powerful boost that we get in that step.

But then what do you do? Given your three options that you have instead of one option, how do you prove to yourself which one of those is the best? I don't have a confident answer to that. I expect this is where the good old fashioned usability testing comes in.

More on prototyping later on:

46:35—Throughout my entire career, my superpower has been prototyping. I've been very quick at knocking out working prototypes of things. I'm the person who can show up at a meeting and say, look, here's how it could work. And that was kind of my unique selling point. And that's gone. Anyone can do what I could do.

## This stuff is exhausting #

26:25—I'm finding that using coding agents well is taking every inch of my 25 years of experience as a software engineer, and it is mentally exhausting. I can fire up four agents in parallel and have them work on four different problems. And by like 11 AM, I am wiped out for the day. [...]

There's a personal skill we have to learn in finding our new limits—what's a responsible way for us not to burn out.

I've talked to a lot of people who are losing sleep because they're like, my coding agents could be doing work for me. I'm just going to stay up an extra half hour and set off a bunch of extra things... and then waking up at four in the morning. That's obviously unsustainable. [...]

There's an element of sort of gambling and addiction to how we're using some of these tools.

## Interruptions cost a lot less now #

45:16—People talk about how important it is not to interrupt your coders. Your coders need to have solid two to four hour blocks of uninterrupted work so they can spin up their mental model and churn out the code. That's changed completely. My programming work, I need two minutes every now and then to prompt my agent about what to do next. And then I can do the other stuff and I can go back. I'm much more interruptible than I used to be.

## My ability to estimate software is broken #

28:19—I've got 25 years of experience in how long it takes to build something. And that's all completely gone—it doesn't work anymore because I can look at a problem and say that this is going to take two weeks, so it's not worth it. And now it's like... maybe it's going to take 20 minutes because the reason it would have taken two weeks was all of the sort of cruffy coding things that the AI is now covering for us.

I constantly throw tasks at AI that I don't think it'll be able to do because every now and then it does it. And when it doesn't do it, you learn, right? But when it *does* do something, especially something that the previous models couldn't do, that's actually cutting edge AI research.

And a related anecdote:

36:56—A lot of my friends have been talking about how they have this backlog of side projects, right? For the last 10, 15 years, they've got projects they never quite finished. And some of them are like, well, I've done them all now. Last couple of months, I just went through and every evening I'm like, let's take that project and finish it. And they almost feel a sort of sense of loss at the end where they're like, well, okay, my backlog's gone. Now what am I going to build?

## It's tough for people in the middle #

29:29—So ThoughtWorks, the big IT consultancy, did an offsite about a month ago, and they got a whole bunch of engineering VPs in from different companies to talk about this stuff. And one of the interesting theories they came up with is they think this stuff is really good for experienced engineers, like it amplifies their skills. It's really good for new engineers because it solves so many of those onboarding problems. The problem is

the people in the middle. If you're mid-career, if you haven't made it to sort of super senior engineer yet, but you're not sort of new either, that's the group which is probably in the most trouble right now.

I mentioned [Cloudflare hiring 1,000 interns](#), and Shopify too.

Lenny asked for my advice for people stuck in that middle:

31:21—That's a big responsibility you're putting on me there! I think the way forward is to lean into this stuff and figure out how do I help this make me better?

A lot of people worry about skill atrophy: if the AI is doing it for you, you're not learning anything. I think if you're worried about that, you push back at it. You have to be mindful about how you're applying the technology and think, okay, I've been given this thing that can answer any question and *often* gets it right. How can I use this to amplify my own skills, to learn new things, to take on much more ambitious projects? [...]

33:05—Everything is changing so fast right now. The only universal skill is being able to roll with the changes. That's the thing that we all need.

The term that comes up most in these conversations about how you can be great with AI is *agency*. I think agents have no agency at all. I would argue that the one thing AI can never have is agency because it doesn't have human motivations.

So I'd say that's the thing is to invest in your own agency and invest in how to use this technology to get better at what you do and to do new things.

## It's harder to evaluate software #

The fact that it's so easy to create software with detailed documentation and robust tests means it's harder to figure out what's a credible project.

37:47 Sometimes I'll have an idea for a piece of software, Python library or whatever, and I can knock it out in like an hour and get to a point where it's got documentation and tests and all of those things, and it looks like the kind of software that previously I'd have spent several weeks on—and I can stick it up on GitHub

And yet... I don't believe in it. And the reason I don't believe in it is that I got to rush through all of those things... I think the quality is probably good, but I haven't spent enough time with it to feel confident in that quality. Most importantly, *I haven't used it yet*.

It turns out when I'm using somebody else's software, the thing I care most about is I want them to have used it for months.

I've got some very cool software that I built that I've *never used*. It was quicker to build it than to actually try and use it!

## The misconception that AI tools are easy #

41:31—Everyone's like, oh, it must be easy. It's just a chat bot. It's not easy. That's one of the great misconceptions in AI is that using these tools effectively is easy. It takes a lot of practice and it takes a lot of trying things that didn't work and trying things that did work.

## Coding agents are useful for security research now #

19:04—In the past sort of three to six months, they've started being credible as security researchers, which is sending shockwaves through the security research industry.

See Thomas Ptacek: [Vulnerability Research Is Cooked](#).

At the same time, open source projects are being bombarded with junk security reports:

20:05—There are these people who don't know what they're doing, who are asking ChatGPT to find a security hole and then reporting it to the maintainer. And the report looks good. ChatGPT can produce a very well formatted report of a vulnerability. It's a total waste of time. It's not actually verified as being a real problem.

A good example of the right way to do this is [Anthropic's collaboration with Firefox](#), where Anthropic's security team *verified* every security problem before passing them to Mozilla.

## OpenClaw #

Of course we had to talk about OpenClaw! Lenny had his running on a Mac Mini.

1:29:23—OpenClaw demonstrates that people want a personal digital assistant so much that they are willing to not just overlook the security side of things, but also getting the thing running is not easy. You’ve got to create API keys and tokens and install stuff. It’s not trivial to get set up and hundreds of thousands of people got it set up. [...]

The first line of code for OpenClaw was written on November the 25th. And then in the Super Bowl, there was an ad for AI.com, which was effectively a vaporware white labeled OpenClaw hosting provider. So we went from first line of code in November to Super Bowl ad in what? Three and a half months.

I continue to love Drew Breunig’s description of OpenClaw as a digital pet:

A friend of mine said that OpenClaw is basically a Tamagotchi. It’s a digital pet and you buy the Mac Mini as an aquarium.

## Journalists are good at dealing with unreliable sources #

In talking about my explorations of AI for data journalism through Datasette:

1:34:58—You would have thought that AI is a very bad fit for journalism where the whole idea is to find the truth. But the flip side is journalists deal with untrustworthy sources all the time. The art of journalism is you talk to a bunch of people and some of them lie to you and you figure out what’s true. So as long as the journalist treats the AI as yet another unreliable source, they’re actually better equipped to work with AI than most other professions are.

## The pelican benchmark #

Of course we talked about pelicans riding bicycles:

56:10—There appears to be a very strong correlation between how good their drawing of a pelican riding a bicycle is and how good they are at everything else. And nobody can explain to me why that is. [...]

People kept on asking me, what if labs cheat on the benchmark? And my answer has always been, really, all I want from life is a really good picture of a pelican riding a bicycle. And if I can trick every AI lab in the world into cheating on benchmarks to get it, then that just achieves my goal.

59:56—I think something people often miss is that this space is inherently funny. The fact that we have these incredibly expensive, power hungry, supposedly the most advanced computers of all time. And if you ask them to draw a pelican on a bicycle, it looks like a five-year-old drew it. That's really funny to me.

## And finally, some good news about parrots #

Lenny asked if I had anything else I wanted to leave listeners with to wrap up the show, so I went with the best piece of news in the world right now.

1:38:10—There is a rare parrot in New Zealand called the Kākāpō. There are only 250 of these parrots left in the world. They are flightless nocturnal parrots—beautiful green dumpy looking things. And the good news is they're having a fantastic breeding season in 2026,

They only breed when the Rimu trees in New Zealand have a mass fruiting season, and the Rimu trees haven't done that since 2022—so there has not been a single baby kākāpō born in four years.

This year, the Rimu trees are in fruit. The kākāpō are breeding. There have been dozens of new chicks born. It's a really, really good time. It's great news for rare New Zealand parrots and you should look them up because they're delightful.

Everyone should [watch the live stream of Rakiura on her nest with two chicks!](#)

## YouTube chapters #

Here's the full list of chapters Lenny's team defined for the YouTube video:

- 00:00: Introduction to Simon Willison
- 02:40: The November 2025 inflection point
- 08:01: What's possible now with AI coding
- 10:42: Vibe coding vs. agentic engineering
- 13:57: The dark-factory pattern
- 20:41: Where bottlenecks have shifted

- [23:36](#): Where human brains will continue to be valuable
- [25:32](#): Defending of software engineers
- [29:12](#): Why experienced engineers get better results
- [30:48](#): Advice for avoiding the permanent underclass
- [33:52](#): Leaning into AI to amplify your skills
- [35:12](#): Why Simon says he's working harder than ever
- [37:23](#): The market for pre-2022 human-written code
- [40:01](#): Prediction: 50% of engineers writing 95% AI code by the end of 2026
- [44:34](#): The impact of cheap code
- [48:27](#): Simon's AI stack
- [54:08](#): Using AI for research
- [55:12](#): The pelican-riding-a-bicycle benchmark
- [59:01](#): The inherent ridiculousness of AI
- [1:00:52](#): Hoarding things you know how to do
- [1:08:21](#): Red/green TDD pattern for better AI code
- [1:14:43](#): Starting projects with good templates
- [1:16:31](#): The lethal trifecta and prompt injection
- [1:21:53](#): Why 97% effectiveness is a failing grade
- [1:25:19](#): The normalization of deviance
- [1:28:32](#): OpenClaw: the security nightmare everyone is looking past
- [1:34:22](#): What's next for Simon
- [1:36:47](#): Zero-deliverable consulting

- [1:38:05](#): Good news about Kakapo parrots

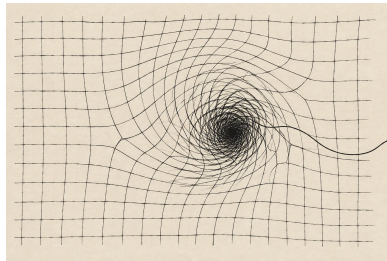
---

# the senior engineer death spiral

SUNIL PAI · 19 SEP 2026 · [SOURCE](#)

---

*(a friend recently got a new job, a very senior role, very well paid, and pretty different from his previous gig. he was asking me how to do a 60 to 80-hour week, how to do well enough to get promoted, etc. and as we talked, I realised he was feeling a bit of imposter syndrome and really wanted to prove himself. I ended up giving him a bit of a monologue. this is more or less what I told him.)*



A hand-drawn grid spiraling into a dense knot, with one loose line escaping to the edge.

okay, so this is the most common failure mode I've seen. I call it the senior engineer death spiral. it usually happens when an engineer goes into a new job, gets a promotion, or even just gets a big project at work. or they ask for a big project because they think to themselves, "oh, if I work harder, do a bigger thing, then I will be rewarded with promotions and what have you," right? like, that's the move. and the thing they do is they tell themselves they need to almost cosplay being a more senior engineer than they are. they're like, "oh, you know what, I'm going to try designing something way more ambitious."

and what happens is they start disappearing for longer periods of time. like, for two, three weeks, you won't hear anything from them. occasionally, during the standup, they'll give what I call the positive update. "yeah, things are going well, you guys. I'll have something to show you quite soon. if you have any questions, reach out." and no one usually reaches out, but they won't really have anything to show.

and in the back of their head, right, they've started the spiral already. they're telling themselves, "oh my god, I haven't really shipped anything in a while. what I need to do is work harder. I'll just work hard enough. I'll save this and no one will know." like, "in the next week, I'll do a whole month's work. and no one will know." they'll start losing sleep, all of that. and it sucks, because bro, they start getting depressed, missing meals. their relationships suffer, both work and personal. and they just start spiraling, spiraling, and it ends up in worst-case scenarios. they burn out, they need to take a month off, they get PIP'd, they get fired, they quit their jobs because they think, "you know what, this is not salvageable." and it really sucks.

and by the way, this isn't something that I've noticed only with other people. this has happened to me. and it's happened so many times that I now recognize it early enough to take a step back and fix it.

see, man, the bigger point here is that because of COVID and coding agents and remote work, things that have happened in the last five to ten years, right, you don't really have the same structure of working next to someone and being given Jira tasks, etc. you have a lot more ownership, and you are unfortunately a little more siloed. so you never want to be in a situation where people don't know what's up with you. like, they shouldn't be saying, "yeah, I don't know what Sunil's been doing," and starting to get concerned, because they might only tell you when it's too late. you want to be in a position where "Sunil doesn't shut the fuck up about what he's working on," right? again, you don't have to be obnoxious about it. you have to share your work.

I think in moments like this, the thing you need to do is actually a little counterintuitive. but you have to start from a position of assuming that everyone around you is operating in good faith, okay? like, they hired you for you. they didn't hire you for who you think you're going to be in six months. they like you for who you are right now, bro.

so the thing to do is actually to drop a level. not to try to be a higher-level engineer, but to almost drop a level and simply become the greatest teammate for a while. you're still responsible for your work, you're just trying to get moving again. you know, "I'm just here to help and support my teammates. I'm going to do bugs, the annoying things on somebody else's plate or that no one has been able to get to. do grunt work, organizational stuff, write-ups. just things to help people out."

and the reason you want to do this is you want to move away from an outcome-based mindset to momentum-based. like, you want to build a routine, because momentum is everything in this, and relationships are an even bigger thing in this. and by getting momentum going and fixing your relationships with other people, the thing you get is stamina. and that's when you realize

the real lesson, which is that big projects are not done with big efforts. it's a big marathon, just slow, steady, incremental work. forget about a weekly basis, like a daily basis, an hourly basis, to the point that it's muscle memory.

you're in automatic mode. you wake up, you make yourself coffee, you get to work, and you just do the work. and then 30 days later, you look back and you see the immense volume of work that you've done just by doing a little bit every day. because the thing you're going for is reliability, fixing your relationships. you want to take the burden off your teammates and your manager, because then, once you build that trust, they will give you bigger work. that's the move here.

remember, you are in the reputation-building business, and software is actually downstream of that. godspeed and best of luck.

---

## [Daily article] September 23: Mary Mallon

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 23 SEP 2026 · [SOURCE](#)

---

NOTE: The bot that posts to this mailing list daily needs a new maintainer. Without a new maintainer, the bot will stop posting to this mailing list daily around September 30, 2026. <https://meta.wikimedia.org/wiki/Talk:daily-article-l>

Mary Mallon (September 23, 1869 – November 11, 1938), dubbed Typhoid Mary, was an Irish-born cook who lived in the US and is believed to have infected up to 57 people with the bacteria that cause typhoid fever, with three confirmed deaths. Between 1897 and 1907, Mallon worked at several homes; residents at four of them contracted typhoid. In 1907, she was forcibly quarantined at Riverside Hospital, but was freed in 1910, promising not to work as a cook. A further outbreak at the Sloane Maternity Hospital, where she was working, resulted in 25 cases and two deaths in early 1915. She was again confined, remaining so until her death in 1938. Her nickname became a term for anyone who spreads disease. Mallon's case raises questions about the balance between individual liberty and public health. No other carrier of typhoid was confined for as long as she was. Her story has been reevaluated, particularly during outbreaks of HIV/AIDS, tuberculosis, and COVID-19. (Full article...).

Read more: [https://en.wikipedia.org/wiki/Mary\\_Mallon](https://en.wikipedia.org/wiki/Mary_Mallon)

---

Today's selected anniversaries:

1884:

The French steamship Arctique ran aground on the northern coast of Cape Virgenes in Argentina; gold was discovered during the rescue effort, triggering the Tierra del Fuego Gold Rush. [https://en.wikipedia.org/wiki/Tierra\\_del\\_Fuego\\_Gold\\_Rush](https://en.wikipedia.org/wiki/Tierra_del_Fuego_Gold_Rush)

1983:

Flying from Pakistan, Gulf Air Flight 771 was blown up by the Abu Nidal Organization while on approach over the United Arab Emirates (UAE), killing all 112 people on board. [https://en.wikipedia.org/wiki/Gulf\\_Air\\_Flight\\_771](https://en.wikipedia.org/wiki/Gulf_Air_Flight_771)

1997:

Irish rock band U2 performed in Sarajevo, the first major artist to hold a concert in the city since the end of the Bosnian War. [https://en.wikipedia.org/wiki/U2\\_concert\\_in\\_Sarajevo](https://en.wikipedia.org/wiki/U2_concert_in_Sarajevo)

2016:

Following a number of high-profile sexual assaults, major reforms were enacted to strengthen laws related to rape in Germany. [https://en.wikipedia.org/wiki/Rape\\_in\\_Germany](https://en.wikipedia.org/wiki/Rape_in_Germany)

---

Wiktionary's word of the day:

Neptune: 1. (Roman mythology) The Roman god of earthquakes and the ocean, the counterpart of Poseidon in Greek mythology. 2. (by extension) 3. A personification of the ocean. 4. (astronomy) The eighth and furthest planet in the Solar System in terms of average distance from the Sun; an ice giant, by size it is the fourth largest planet. [...] 5. (nautical, historical) A sailor acting as Neptune (proper noun sense 1), the Roman god of the ocean, during a ceremony on board a ship when it is crossing the equator. 6. About Word of the Day 7. Nominate a word 8. Leave feedback <https://en.wiktionary.org/wiki/Neptune>

---

Wikiquote quote of the day:

The gentle rain falls softly on my weary eyes  
As if to hide a lonely tear  
My life will be forever autumn  
Now you're not here. --Gary Osborne [https://en.wikiquote.org/wiki/Gary\\_Osborne](https://en.wikiquote.org/wiki/Gary_Osborne)